# HASBoost: History-based Adaptive Sketching for Fast and Accurate Gradient Boosting


## Abstract

Gradient boosting machines achieve state-of-the-art performance on tabular data but face high computational costs in high-dimensional and multi-label settings. We propose HASBoost, an adaptive algorithm that reduces split-search cost within the sequential boosting framework for multi-output problems, improving the practical accuracy–speed trade-off in high-cost regimes. The primary contribution of HASBoost is a training-aware optimization method that uses historical statistics to gate approximation and adapt row-column sketch allocation across optimization regimes. We provide a theoretical interpretation of the resulting implicit regularization and show that sketched split scores incur a controlled approximation error in stable regimes. Empirical evaluation on standard multi-label benchmarks shows dataset-dependent behavior: HASBoost provides substantial training-time reductions in larger/high-cost settings while matching or improving F1-score in most cases, and can be slower on smaller settings where bookkeeping overhead dominates. Compared to SketchBoost—a strong recent baseline for scalable multi-output gradient boosting—HASBoost often improves training time, while inference-time effects vary by dataset. Empirical evaluation across the final benchmark suite (age_prediction, cifar10, genbase, mediamill, mnist, yeast, MoA, Delicious) shows strongly regime-dependent behavior: in high-cost regimes (especially mediamill and mnist), HASBoost variants yield large training-time reductions while preserving or improving F1, whereas on smaller/lower-cost regimes the speed gains are less uniform and may partially reverse depending on the variant. Our work connects adaptive sketching with ensemble methods, suggesting new avenues for scalable and theoretically principled boosting in high-dimensional output spaces.



.**AUTHORERR: Missing \icmlcorrespondingauthor.**



Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

**Keywords:** gradient boosting, computational efficiency, adaptive sampling, multi-label classification.


## 1. Introduction

Gradient boosting machines (GBMs), a boosting-based ensemble method distinct from bagging (Breiman, 1996), are among the most effective methods for modeling tabular and heterogeneous data, consistently outperforming deep neural networks in a wide range of applications, from recommendation systems to financial forecasting (Chen & Guestrin, 2016; Ke et al., 2017; Prokhorenkova et al., 2018). However, their computational demands escalate significantly with increasing data dimensionality, particularly in high-dimensional multi-output scenarios such as multi-label classification or multi-target regression, where the number of features, samples, or outputs can reach thousands. This bottleneck arises from the sequential nature of ensemble construction, where each weak learner (typically a decision tree) is fitted to the residuals of the previous model, requiring repeated computations over the full gradient and Hessian matrices. In modern applications—including real-time inference on edge devices, large-scale online learning, and resource-constrained environments—there is a pressing need for acceleration techniques that maintain predictive accuracy without prohibitive computational overhead.

Existing approaches to mitigate these costs include instance subsampling methods like Gradient-based One-Side Sampling (GOSS) in LightGBM (Ke et al., 2017), which prioritizes high-gradient samples, or Stochastic Gradient Boosting (Friedman, 2002), which uses random subsets. Other strategies focus on model regularization, such as Dropout at Random Trees (DART) (Rashmi & Gilad-Bachrach, 2015) which introduces dropout to combat overfitting, or explicit penalties like those in AdaBoost (Freund & Schapire, 1997). However, these methods often involve trade-offs: fixed or static sampling can degrade accuracy by ignoring task-specific dynamics, especially in multi-output settings where outputs exhibit varying complexities and correlations; regularization primarily targets generalization rather than runtime efficiency; and the inherent sequential dependency of boosting remains a fundamental limiter.

To address these limitations, we propose HASBoost (History-based Adaptive Sketching for Boosting), a training-aware optimization method for multi-output boosting instantiated via adaptive sketching. Instead of using a single static approximation policy, HASBoost uses optimization-history signals to control (i) when approximation should be activated, (ii) how row/column sketch budget should be reallocated across optimization regimes, and (iii) how prediction-level bias is controlled through full-statistics leaf recomputation. This design focuses computation on informative regions while keeping approximation conservative when the trajectory is unstable.

Our key contributions are as follows:

- A training-aware optimization method for multi-output GBDT, with three coupled control components: stability-gated activation, history-guided row/column budget allocation via the sample–output weight matrix $W^{(t)}$, and full-statistics leaf recomputation for prediction-level bias control.
- A theoretical analysis providing an interpretation of the implicit regularization induced by history-guided sketching, together with a simple error bound for sketched split scores and an unbiased alternative estimator.
- An empirical evaluation on standard one-label and multi-label benchmarks (e.g., *MediaMill*), showing strong speed–quality trade-offs in high-cost regimes (including training speedups up to 9.6× over XGBoost), while explicitly documenting heterogeneous behavior on smaller problems and dataset-dependent inference-time effects. Compared to SketchBoost, HASBoost often reduces training time while matching or improving F1-score.

The remainder of this paper is organized as follows: Section 2 discusses related work, Section 3 details the HASBoost algorithm, Section 5 provides theoretical analysis, Section 6 reports experimental setups, Section 7 provide analysis of metrics, and Section 9 discusses extensions and concludes.

## 2. Related Work

**Gradient boosting for tabular learning.** Gradient boosting (GB), formalized as functional gradient descent, remains a dominant paradigm for tabular and structured data due to its strong accuracy–efficiency trade-offs (Friedman, 2001; Mason et al., 1999). Industrial-grade systems such as XGBoost, LightGBM, and CatBoost refine this core recipe via second-order objectives, histogram-based split search, GPU support, and robust handling of sparsity and categorical features (Chen & Guestrin, 2016; Ke et al., 2017; Prokhorenkova et al., 2018). Despite major engineering improvements, classical boosting remains fundamentally sequential: each stage depends on residuals from the previous model, which limits parallelism and makes training costly in high-dimensional and multi-output regimes.

**Sampling and sketching for faster tree construction.** A large body of work accelerates boosting by reducing the cost of split evaluation through sampling, subsampling, or approximate statistics. Stochastic Gradient Boosting subsamples instances per iteration to lower compute while preserving generalization (Friedman, 2002). LightGBM popularized Gradient-based One-Side Sampling (GOSS), prioritizing high-gradient samples to speed up training (Ke et al., 2017). For multi-output settings, SketchBoost reduces split-search overhead by sketching gradient/Hessian statistics, yielding substantial speedups in multi-label problems while keeping accuracy competitive (Iosipoi et al., 2022). However, most existing schemes are either static (fixed rates) or rely on instantaneous signals (e.g., current gradient magnitude), which can be brittle early in training when optimization dynamics are volatile.

**Non-stationarity and distributed constraints.** Recent work increasingly highlights that boosting systems must handle non-stationary and constrained regimes (streaming updates, evolving distributions, or deployment-time constraints), where the timing of approximation and updates matters. In a complementary direction, federated and privacy-preserving boosting emphasizes efficiency under distributed constraints (communication, partial visibility), motivating algorithms that adaptively prioritize computation where it is most informative (Lindskog et al., 2023; Pezoulas et al., 2024; Zhang et al., 2025). These settings reinforce a key design choice in HASBoost: approximation should be activated *selectively*, only after the training trajectory reaches a sufficiently stable regime.

**Regularization, pruning, and compact tree ensembles** Beyond speeding up training, recent efforts focus on reducing ensemble size and improving deployability without sacrificing accuracy. Functionally-identical pruning shows that boosted tree ensembles can often be simplified *without changing predictions*, offering strong compression while keeping fidelity exact (Emine et al., 2025). Related lines of work explore verification-friendly tree ensembles for reliable deployment and auditing (Calzavara et al., 2025). These results align with the empirical effect we observe in HASBoost: by injecting controlled approximation only in stable phases, the method implicitly discourages late-stage, high-variance micro-corrections and tends to yield simpler ensembles (e.g., shallower trees or fewer effective splits), consistent with our interpretation of sketching as an implicit regularizer.

*Table 1.* Positioning of HASBoost as a training-aware optimization layer.

| Method | When approximation is applied | Allocation signal | Leaf-stage bias control |
|---|---|---|---|
| GOSS (Ke et al., 2017) | Per-iteration row sampling (no explicit stability-gated switching) | Instantaneous gradient magnitude (rows) | No dedicated decoupled recomputation step |
| SketchBoost (Iosipoi et al., 2022) | Approximate split search with preset sketch budget per run | Preset sketch rule/size | Full-statistics leaf values after structure |
| HASBoost (ours) | Stability-gated switching between full and approximate split search | History-guided adaptive row/column budget by regime | Full-statistics leaf recomputation for all outputs |

**Positioning of HASBoost.** In summary, HASBoost is closest in spirit to sketching-based multi-output boosting (Iosipoi et al., 2022), but differs in *how and when* approximation is applied: we use history-guided dynamics tracking to (i) detect stable regimes and (ii) prioritize rows/columns for sketching via adaptive importance weights. This bridges acceleration methods (sampling/sketching) with recent emphasis on deployability and compactness (pruning/verification) (Emine et al., 2025; Calzavara et al., 2025), and with modern constraints in distributed settings (Lindskog et al., 2023; Pezoulas et al., 2024; Zhang et al., 2025).

**Training-aware optimization view.** We frame HASBoost as a training-aware optimization method inside the standard boosting pipeline, rather than as a new boosting paradigm. In this view, sketching is the approximation operator, while the main contribution is the control layer: regime detection, adaptive budget allocation, and bias control. Table 1 summarizes this positioning relative to representative prior approaches.

# 3. Preliminary

## 3.1. Problem Definition

In multioutput prediction tasks, such as multiclass classification, multilabel classification, or multioutput regression, the goal is to learn a model that maps input features $\mathbf{x}_i \in \mathbb{R}^m$ to high-dimensional outputs $\mathbf{y}_i \in \mathbb{R}^d$ for a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$, where $d$ is large. Gradient Boosted Decision Trees (GBDT) achieve state-of-the-art performance on tabular data but scale poorly with $d$, as training time grows linearly with output dimensionality due to exhaustive scoring during tree structure search.

## 3.2. Gradient Boosting Formulation

GBDT constructs an additive ensemble $F_t = F_{t-1} + \varepsilon f_t$, where $f_t : \mathbb{R}^m \to \mathbb{R}^d$ are decision trees serving as base learners, and $\epsilon > 0$ is a learning rate. At iteration $t$, $f_t$ minimizes the second-order Taylor approximation of a loss $l : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$:

$$f_t^* \in \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \left[ \mathbf{g}_i^\top f(\mathbf{x}_i) + \frac{1}{2} f(\mathbf{x}_i)^\top h_i f(\mathbf{x}_i) \right] + \Omega(f), \tag{1}$$

where $\mathcal{F}$ is the class of decision trees, $\Omega(f)$ is a regularizer (e.g., $\ell_2$ on leaf values), and the gradients and Hessians are:

$$\mathbf{g}_i = \nabla_a l(\mathbf{y}_i, a)\Big|_{a=F_{t-1}(\mathbf{x}_i)}, \quad h_i = \nabla_a^2 l(\mathbf{y}_i, a)\Big|_{a=F_{t-1}(\mathbf{x}_i)}. \tag{2}$$

For multioutput trees, Hessians are often diagonalized for efficiency, yielding per-output components $g_i^k$ and $h_i^k$ for $k = 1, \ldots, d$ forming the gradient matrix $G^{(t)} \in \mathbb{R}^{n \times d}$ and Hessian matrix $H^{(t)} \in \mathbb{R}^{n \times d}$ over the given dataset. Tree construction splits leaves to maximize information gain via a scoring function $S(R) = \sum_{j=1}^{d} \frac{\left(\sum_{\mathbf{x}_i \in R} g_i^j\right)^2}{\sum_{\mathbf{x}_i \in R} h_i^j + \lambda}$, but this incurs $O(d)$ overhead per split evaluation.

To address scalability, sketched approximations reduce the effective dimensionality during split search (e.g., via top-$k$ outputs, random sampling, or projections) while computing full leaf values post-structure.

## 3.3. Finding the Tree Structure

In multioutput GBDT using the single-tree strategy, tree construction proceeds greedily: starting from the root, each step selects the split that maximizes information gain across existing leaves.

For a leaf region $R$, the scoring function is

$$S(R) = \sum_{k=1}^{d} \frac{\left(\sum_{i \in R} g_i^k\right)^2}{\sum_{i \in R} h_i^k + \lambda}, \tag{3}$$

where $g_i^k$ and $h_i^k$ are the $k$-th components of the per-sample gradient and (diagonal) Hessian, and $\lambda > 0$ is $\ell_2$-regularization on leaf values.

The information gain for splitting $R$ into $R_{\text{left}}$ and $R_{\text{right}}$ is

$$\text{Gain} = \frac{1}{2}\left[S(R_{\text{left}}) + S(R_{\text{right}}) - S(R)\right]. \tag{4}$$

**Second-order leaf values.** Once the tree structure is fixed, the optimal (regularized) leaf value for each output $k$ on a leaf region $R$ under the diagonal-Hessian approximation is

$$w_R^k = -\frac{\sum_{i \in R} g_i^k}{\sum_{i \in R} h_i^k + \lambda}, \qquad k = 1, \ldots, d. \tag{5}$$

We compute these leaf values using the full (unsketched) statistics to avoid approximation bias at the prediction level.

# 4. Methodology

## 4.1. History Gathering

**Definition 4.1** (Optimization History)**.** At boosting iteration $t$, let $G^{(t)} \in \mathbb{R}^{n\times d}$ and $H^{(t)} \in \mathbb{R}^{n\times d}$ denote the per-sample (diagonal) gradient and Hessian matrices. We maintain two lightweight history signals:

**(i) Per-sample scale trackers (used for adaptive weights).** We track per-sample magnitudes

$$s_i^{(t)} := \left\|G_{i,:}^{(t)}\right\|_2, \qquad u_i^{(t)} := \left\|H_{i,:}^{(t)}\right\|_2, \tag{6}$$

and update their exponentially-smoothed per-sample histories

$$\mathbf{h}_g^{(t)} := \alpha\,\mathbf{h}_g^{(t-1)} + (1-\alpha)\,\mathbf{s}^{(t)}, \mathbf{h}_h^{(t)} := \alpha\,\mathbf{h}_h^{(t-1)} + (1-\alpha)\,\mathbf{u}^{(t)}, \tag{7}$$

where $\mathbf{s}^{(t)} := (s_1^{(t)}, \ldots, s_n^{(t)})^\top$ and $\mathbf{u}^{(t)} := (u_1^{(t)}, \ldots, u_n^{(t)})^\top$.

**(ii) A global volatility statistic (used for the scheduler).** We measure short-term change via

$$\Delta\mathbf{s}^{(t)} := \mathbf{s}^{(t)} - \mathbf{s}^{(t-1)}, \qquad \Delta\mathbf{u}^{(t)} := \mathbf{u}^{(t)} - \mathbf{u}^{(t-1)}, \tag{8}$$

## 4.2. Sketch Scheduler: Detecting Stable Optimization Regimes

Gradient and Hessian values are typically noisy and uninformative during the early boosting iterations, when the model is rapidly changing and the loss surface is highly non-convex. Applying sketching or subsampling too early can introduce excessive bias or variance, degrading final performance. Therefore, we delay activation of approximation techniques until the training dynamics enter a *stable regime*, where further updates are comparatively small and local (see Fig. 1).

To detect this transition automatically, we use an online change-point heuristic built on Definition 4.1. The scheduler activates sketching (and/or instance subsampling) at iteration $t$ if the relative volatility drops below a threshold:

$$\frac{\|\Delta\mathbf{s}^{(t)}\|_1}{\|\mathbf{h}_g^{(t-1)}\|_1 + \varepsilon} < \theta, \tag{9}$$

where $\theta \in (0, 1)$ is a threshold (e.g., initialized to 0.1–0.3) and $\varepsilon > 0$ prevents division by zero.

Intuitively, early iterations exhibit large $\Delta\mathbf{s}^{(t)}$ relative to the historical scale $\mathbf{h}_g^{(t-1)}$, while stable phases show small relative changes. The scheduler is computationally negligible

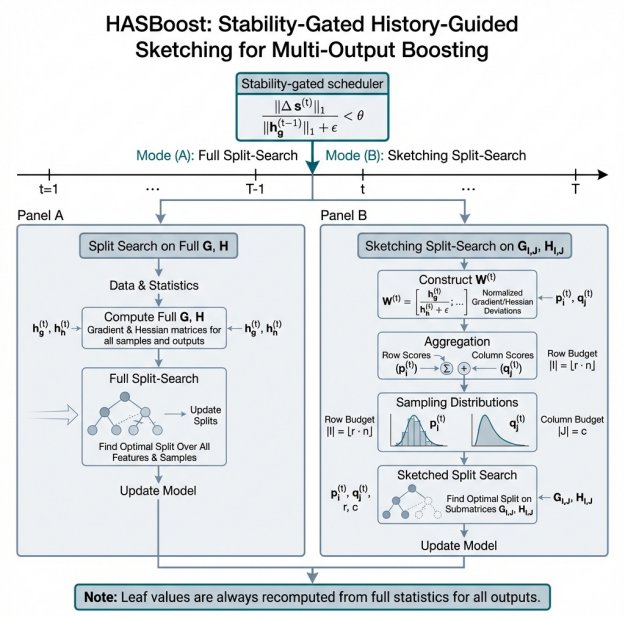


*Figure 1.* Conceptual plot of gradient norm evolution during boosting. The curve shows initial high-variance fluctuations followed by a stable regime. A sliding window (shaded) represents the historical values affecting the current mean the most, and sampling is triggered when the smoothed gradient norm falls below the stability threshold (dashed).

($O(n)$ per iteration) and triggers approximation only when the optimization trajectory becomes sufficiently stable.

**Definition 4.2** (Complexity-Aware Adaptive Sampling Weights)**.** At boosting iteration $t$, we maintain two per-sample history vectors $\mathbf{h}_g^{(t)} \in \mathbb{R}^n$ and $\mathbf{h}_h^{(t)} \in \mathbb{R}^n$ that track exponentially smoothed gradient- and Hessian-scale proxies (Definition 4.1). We use per-sample (rather than per-output) normalization to keep the history state $O(n)$ even when $d$ is large, and to provide a stable scale reference shared across all outputs of the same training instance.

We define a weight matrix $W^{(t)} \in \mathbb{R}^{n\times d}$ whose entries $W_{ij}^{(t)}$ serve as adaptive importance scores for sample $i$ and output dimension $j$. The entry is computed element-wise as

$$W_{ij}^{(t)} = \phi\Big(\underbrace{\Big(\frac{G_{ij}^{(t)}}{|h_{g,i}^{(t-1)}| + \varepsilon} - 1\Big)}_{\text{normalized gradient deviation}} \cdot \underbrace{\Big(\frac{H_{ij}^{(t)}}{|h_{h,i}^{(t-1)}| + \varepsilon} - 1\Big)}_{\text{normalized Hessian deviation}}\Big), \tag{10}$$

where $\varepsilon > 0$ is a small stabilization constant (typically $10^{-6}$–$10^{-4}$), and $\phi : \mathbb{R} \to [0, 1]$ is a monotonically non-increasing transformation.

We investigate two concrete choices for the transformation $\phi$:

- **Hyperbolic (greedy)**: $\phi(x) = \mathrm{MinMax}(-x)$

This produces a sharp, almost step-like assignment: high weights are concentrated on the most deviant (complex) rows and columns, leading to a greedy selection strategy.

- **Sigmoidal (balanced)**: $\phi(x) = 1 - \sigma(x) = 1 - \big(1 + \exp(-x)\big)^{-1}$

  This yields a smoother, more uniform distribution of weights among moderately-to-highly informative directions, reducing the risk of over-focusing on outliers.

Here $\mathrm{MinMax}(\cdot)$ denotes an affine rescaling to $[0, 1]$ applied over all entries of its argument at iteration $t$. This history-dependent weighting mechanism implicitly regularizes the model by focusing computation on samples and outputs that deviate most from their recent stable behavior, thereby concentrating learning effort where updates are still impactful. Operationally, $W^{(t)}$ is the interface between optimization dynamics and compute allocation: it induces $p_i^{(t)}$ and $q_j^{(t)}$, and thus determines where the fixed sketch budgets $(r, c)$ are spent.

### 4.3. Sketching

After computing the weight matrix $W^{(t)} \in \mathbb{R}^{n\times d}$ (Definition 4.2), we perform *joint row–column sketching* of the current $G^{(t)}$ and $H^{(t)}$. Let $r \in (0, 1]$ be the row sampling fraction and $c \in \{1, \dots, d\}$ the number of sampled output dimensions.

**Joint row–column distribution induced by $W^{(t)}$.** We first aggregate $W^{(t)}$ into nonnegative row/column scores

$$a_i^{(t)} := \sum_{j=1}^{d} W_{ij}^{(t)}, \qquad b_j^{(t)} := \sum_{i=1}^{n} W_{ij}^{(t)}, \tag{11}$$

and define marginal sampling distributions

$$p_i^{(t)} := \frac{a_i^{(t)} + \tau}{\sum_{i'=1}^{n}(a_{i'}^{(t)} + \tau)}, \qquad q_j^{(t)} := \frac{b_j^{(t)} + \tau}{\sum_{j'=1}^{d}(b_{j'}^{(t)} + \tau)}, \tag{12}$$

where $\tau \geq 0$ is an optional smoothing constant to avoid degenerate probabilities.

**Sampling and sketch formation.** We implement joint row–column sketching by sampling rows and columns under the product distribution $\pi_{ij}^{(t)} := p_i^{(t)} q_j^{(t)}$, i.e., selecting both an instance subset and an output subset in the same iteration. Concretely, we sample a row index set $\mathcal{I} \subseteq \{1, \dots, n\}$ of size $|\mathcal{I}| = \lfloor rn \rfloor$ according to $\{p_i^{(t)}\}_{i=1}^{n}$ and a column index set $\mathcal{J} \subseteq \{1, \dots, d\}$ of size $|\mathcal{J}| = c$ according to $\{q_j^{(t)}\}_{j=1}^{d}$. Unless stated otherwise, all experiments use sampling *without replacement* for both rows and columns.

**Default (biased) sketch and use in tree growth.** By default, HASBoost uses the sampled row set $\mathcal{I}$ and column set $\mathcal{J}$ directly (without Horvitz–Thompson reweighting), yielding a simple, low-variance approximation. The sketched matrices are

$$G_S^{(t)} = G_{\mathcal{I},\mathcal{J}}^{(t)}, \qquad H_S^{(t)} = H_{\mathcal{I},\mathcal{J}}^{(t)}. \tag{13}$$

They are used exclusively to evaluate candidate split gains during histogram-based tree growth, reducing split evaluation from $O(nd)$ to $O(|\mathcal{I}|\,|\mathcal{J}|) = O(rnc)$. Once the tree structure is determined, we compute exact leaf values for all $d$ outputs using full (unsketched) statistics (Eq. 5).

**Coupled row–column sampling via leverage scores (HASBoost-SVD).** As an optional variant, we go beyond the product-marginals $\pi_{ij}^{(t)} := p_i^{(t)} q_j^{(t)}$ by exploiting low-rank structure in $W^{(t)}$. Specifically, we compute a rank-$k$ approximation $W^{(t)} \approx U_k \Sigma_k V_k^\top$ (via randomized SVD) and derive leverage-score marginals

$$p_i^{(t)} \propto \|U_k(i,:)\|_2^2, \qquad q_j^{(t)} \propto \|V_k(j,:)\|_2^2, \tag{14}$$

which can better preserve joint sample–output structure when $W^{(t)}$ is approximately low-rank.

Details and ablations—Horvitz–Thompson (unbiased) reweighting, greedy vs. sampling variants, and additional implementation choices (including HASBoost-SVD update frequency)—are deferred to Appendix B.

In both cases, the sketched matrices $G_S^{(t)}$, $H_S^{(t)}$ are used solely to evaluate information gain during the histogram-based split search. Once the tree structure is fixed, the exact leaf values are computed using the full matrices (no approximation at this stage).

The complete procedure, including the conditional history update, adaptive weight computation, joint row-column sketching, and tree construction logic, is summarized in Algorithm 1.

## 5. Theoretical Analysis

### 5.1. Adaptive Sample–Output Clustering

Building on the per-sample optimization history (Definition 4.1), we leverage this accumulated signal to perform adaptive, per-sample weighting during both tree construction and gradient sketching. At each boosting iteration $t$, we compute a scalar *complexity weight* $W_{ij}^{(t)}$ for each sample–output pair $(i, j)$ as defined in Eq. 10. This weight reflects the expected utility of allocating computational resources to that pair. We identify four characteristic clusters (see Fig. 2):

(i) **Low gradient, low Hessian** (*Well-fitted or noise*):

**Algorithm 1** History-based Adaptive Sketching (HASBoost)

**Require:** Dataset $D$, loss $\ell$, iterations $T$, row fraction $r$, column budget $c$, EMA $\alpha$, threshold $\theta$, $\varepsilon$, smoothing $\tau$

Initialize $\mathbf{h}_g^{(0)}, \mathbf{h}_h^{(0)} \leftarrow \mathbf{0}$; $\mathbf{s}^{(0)} \leftarrow \mathbf{0}$; $\mathbf{u}^{(0)} \leftarrow \mathbf{0}$
**for** $t \leftarrow 1$ **to** $T$ **do**
  $G^{(t)}, H^{(t)} \leftarrow CountGH(D, \ell)$
  $\mathbf{s}^{(t)} \leftarrow (\|G_{1,:}^{(t)}\|_2, \ldots, \|G_{n,:}^{(t)}\|_2)^\top$
  $\mathbf{u}^{(t)} \leftarrow (\|H_{1,:}^{(t)}\|_2, \ldots, \|H_{n,:}^{(t)}\|_2)^\top$
  $\Delta\mathbf{s}^{(t)} \leftarrow \mathbf{s}^{(t)} - \mathbf{s}^{(t-1)}$
  $\Delta\mathbf{u}^{(t)} \leftarrow \mathbf{u}^{(t)} - \mathbf{u}^{(t-1)}$
  $\mathbf{h}_g^{(t)} \leftarrow \alpha\, \mathbf{h}_g^{(t-1)} + (1-\alpha)\, \mathbf{s}^{(t)}$
  $\mathbf{h}_h^{(t)} \leftarrow \alpha\, \mathbf{h}_h^{(t-1)} + (1-\alpha)\, \mathbf{u}^{(t)}$
  **if** $\frac{\|\Delta\mathbf{s}^{(t)}\|_1}{\|\mathbf{h}_g^{(t-1)}\|_1+\varepsilon} < \theta$ **then** {Stable regime}
    $W_{ij}^{(t)} \leftarrow \phi\left(\left(\frac{G_{ij}^{(t)}}{|h_{g,i}^{(t-1)}|+\varepsilon} - 1\right) \cdot \left(\frac{H_{ij}^{(t)}}{|h_{h,i}^{(t-1)}|+\varepsilon} - 1\right)\right)$
    $a_i^{(t)} \leftarrow \sum_{j=1}^{d} W_{ij}^{(t)}, \quad b_j^{(t)} \leftarrow \sum_{i=1}^{n} W_{ij}^{(t)}$
    $p_i^{(t)} \leftarrow \frac{a_i^{(t)}+\tau}{\sum_{i'=1}^{n}(a_{i'}^{(t)}+\tau)}, \quad q_j^{(t)} \leftarrow \frac{b_j^{(t)}+\tau}{\sum_{j'=1}^{d}(b_{j'}^{(t)}+\tau)}$
    Sample $\mathcal{I}$ of size $\lfloor rn \rfloor$ using $\{p_i^{(t)}\}$; sample $\mathcal{J}$ of size $c$ using $\{q_j^{(t)}\}$
    $G_S^{(t)} \leftarrow G_{\mathcal{I},\mathcal{J}}^{(t)}, \quad H_S^{(t)} \leftarrow H_{\mathcal{I},\mathcal{J}}^{(t)}$
    Build tree using $G_S^{(t)}$, $H_S^{(t)}$ for split finding
  **else**
    Build tree using full $G^{(t)}$, $H^{(t)}$
  **end if**
  Compute leaf values on full $G^{(t)}$, $H^{(t)}$
  $F_t \leftarrow F_{t-1} + \eta_t h_t$
**end for**

Small residuals with flat curvature. These points contribute minimally to the current Newton direction and can be down-sampled or sketched aggressively with little loss in fidelity.

(ii) **High gradient, low Hessian** (*Active training*): Large residuals with gentle curvature. The model actively trains on these points.

(iii) **Low gradient, high Hessian** (*High-curvature anchors*): Small residuals but high curvature. Predictions are near-optimal but lie in regions of sharp curvature (e.g., near the decision boundary for logistic loss). These samples stabilize training and help refine splits, but require precise rather than large updates.

(iv) **High gradient, high Hessian** (*Volatile errors*): Large residuals with high curvature. Predictions are both incorrect and sensitive to small changes. These points are in active flux; aggressive updates may be beneficial but risk instability.

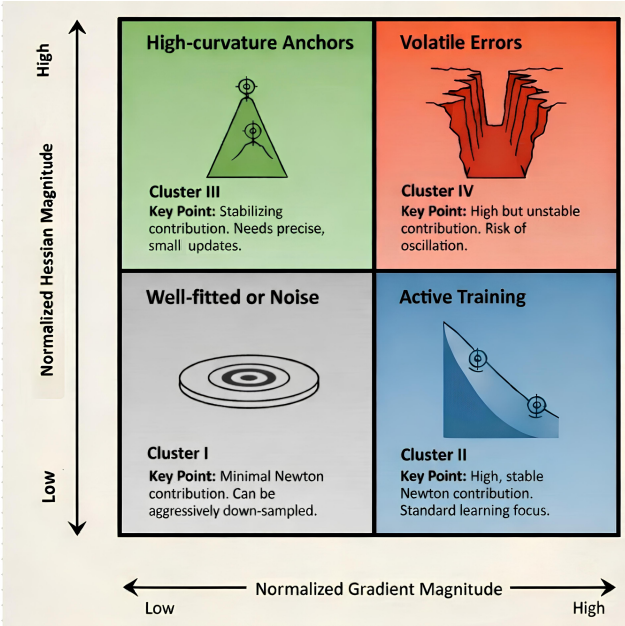


*Figure 2.* The proposed sample–output pairs clusters based on gradient and Hessian change in comparison to their historical values.

This taxonomy directly informs our resource allocation strategy. The potential for model improvement is concentrated in clusters (ii) and (iii): cluster (ii) represents errors where the ensemble can still make gains, while cluster (iii) consists of "anchor" points that help shape the decision boundary. In contrast, cluster (iv) presents a high risk of noisy updates, and cluster (i) offers negligible per-iteration benefit.

We therefore prioritize computation dynamically each iteration, assigning higher complexity weights $w_{ij}^{(t)}$ to sample–output pairs in clusters (ii) and (iii), and lower weights to those in clusters (i) and (iv). The classification itself is dynamic, evolving with the optimization state $F_{t-1}$.

This approach is particularly powerful in multi-output or multi-label settings, where different tasks may converge at different rates. The vector-valued history $\mathbf{h}_v^{(t)}$ enables a joint view across outputs: if a sample exhibits consistent behavior (e.g., "stubborn" or "anchor") across many tasks, we can apply coordinated sketching or sampling, thereby amortizing computational cost without sacrificing task-specific accuracy. When gradients and Hessians are aligned across outputs, we can perform output-compressed updates, drastically reducing the overhead of multi-output tree construction.

### 5.2. Implicit Tikhonov Regularization via Adaptive Sketching

In HASBoost, sketching is activated only after the training dynamics enter a stable regime (Eq. 9). In this phase, row–

column sketching acts as a controlled approximation that suppresses late-stage, high-variance "micro-corrections", yielding an *implicit* regularization effect analogous to early stopping / Tikhonov-style smoothing.

We provide the detailed split-score approximation bounds (biased vs. Horvitz–Thompson), the joint row–column extension, and a sufficient margin condition for preserving the argmax split choice in Appendix A.

## 6. Experiments

**Experimental design.** We evaluate HASBoost on datasets of varying scale to stress both statistical performance and computational efficiency. We use two smaller datasets (*Yeast* for single-label classification and *Birds* for multi-label classification), two larger datasets (*Age Prediction* for single-label classification and *MediaMill* for multi-label classification), and one common image benchmark (*MNIST*, single-label classification). In addition, we include two large-scale multi-label benchmarks, *MoA* and *Delicious*, to further assess performance in high-dimensional output spaces with strong label imbalance. Importantly, *all* datasets are treated and evaluated in a multi-label manner by representing the target as a multi-hot vector (for one-label problems, this reduces to a one-hot encoding).

**Datasets and metadata.** Table 2 summarizes the key dataset statistics used throughout the paper. All tasks are evaluated in a multi-label setting (single-label datasets are converted to one-hot multi-label targets). The *MoA* dataset corresponds to the Mechanism of Action prediction task with a large number of targets (206 labels) and high-dimensional feature space (876 features). The *Delicious* dataset is a classical extreme multi-label benchmark with very high label cardinality (983 labels) and sparse annotations.

*Table 2.* Dataset metadata.

| Dataset | $n$ samples | $d$ classes/labels | $m$ features |
|---|---|---|---|
| *Birds* | 645 | 19 | 264 |
| *Yeast* | 2417 | 14 | 103 |
| *MNIST* (PCA-100) | 70000 | 10 | 100 |
| *Age Prediction* (CoLES-128) | 150000 | 4 | 128 |
| *MediaMill* | 43907 | 101 | 120 |
| *MoA* | 23814 | 206 | 876 |
| *Delicious* | 16105 | 983 | 500 |

**Feature representations.** For MNIST, we extract a 100-dimensional PCA embedding that preserves 90% of the explained variance. For *Age Prediction*, each transaction sequence is mapped to a fixed 128-dimensional embedding using CoLES; the downstream boosting models are trained on these embeddings. For *MoA*, we use the provided high-dimensional tabular features (gene expression and cell viability measurements) without additional feature engineering. For *Delicious*, we use the standard bag-of-words / TF-IDF-style representation commonly adopted in multi-label benchmarks.

**Evaluation protocol.** We report results using 5-fold cross-validation (same folds for all methods). For each fold, we train on 4 folds and evaluate on the held-out fold; the reported metrics and runtimes are averaged over the 5 runs. For large-scale datasets (*MoA* and *Delicious*), we ensure identical splits and preprocessing across all methods to guarantee fair comparison, particularly for runtime measurements.

**Metrics and decision rules.** For quality, we report the weighted F1 score computed on multi-label predictions. For multi-label datasets, we threshold predicted probabilities at $0.5$ (unless stated otherwise) and compute weighted F1 over labels. For single-label datasets converted to one-hot targets, this protocol reduces to standard per-class F1 averaged with class-frequency weights. Due to the large number of labels and strong sparsity in *MoA* and especially *Delicious*, weighted F1 provides a more stable and informative evaluation compared to subset accuracy or exact match metrics.

**Runtime measurement.** For efficiency, we report training time and inference time on the same hardware/software stack (NVIDIA A100-PCIE-40GB, CUDA 13.0). Training time is end-to-end wall-clock time for model fitting (including histogram construction and sketching overhead). Inference time is measured on the full test fold using batch prediction; we report the mean over 10 repeated runs after a short warm-up

**Baselines and fairness.** We compare against XGBoost, LightGBM, and SketchBoost. In our analysis, SketchBoost serves as a no-history sketching reference family (preset sketch policy), which we contrast with history-guided allocation through $W^{(t)}$. To maximize equivalence and ensure fairness, we apply the same preprocessing, folds, and evaluation code for all methods. When comparing methods, we also keep common hyperparameters aligned wherever the API permits. In particular, the learning rate and subsampling ratio are fixed to `0.005` and `0.5`, respectively, across all approaches. Other hyperparameters (e.g., tree depth/leaf constraints, column sketch budget for SketchBoost/HASBoost, and scheduler parameters for HASBoost) are varied within pre-defined grids. All models are tuned under the same cross-validation procedure and the same tuning budget.

**Reproducibility and variance.** To control randomness from sampling and GPU kernels, we fix a global random seed and use deterministic data splits. Unless stated other-

wise, HASBoost uses sampling without replacement and the same sketching parameters across folds, except for the tuned hyperparameters.

## 7. Results

**Main results.** Table 3 summarizes the quality–efficiency trade-off (Table 2 provides dataset scale).

Across the larger/high-cost regimes (notably *MNIST* and *MediaMill*), HASBoost achieves the best (or tied-best) F1 while substantially reducing training time relative to XGBoost and the recent SketchBoost baseline. On smaller-scale problems (*Yeast*, and partially *Birds*), the gains are less consistent: when the per-split cost is already low, the additional overhead of maintaining history and sampling can dominate, leading to less predictable runtime improvements.

**Nonlinearity variants in $W^{(t)}$.** Table 3 includes two HASBoost variants that differ only in the nonlinear map $\phi(\cdot)$ used in Eq. 10: linear and sigmoidal. This comparison isolates how the weighting transformation changes the quality–time trade-off under the same training-aware optimization pipeline.

**Scope of claims.** HASBoost is designed to reduce split-search cost, not to remove the sequential dependency of boosting. Accordingly, we interpret efficiency gains as regime-dependent: they are strongest in high-output/high-cost settings and weaker when fixed overhead dominates. Inference-time behavior is also non-uniform across datasets.

**Why small-scale behavior is less predictable (speculation).** At small $n$ and small $d$, training is dominated by fixed costs (histogram construction, kernel launches/memory movement, and per-iteration bookkeeping) rather than by the $O(nd)$ split-scoring term. In this regime, stochastic row/column selection also has higher relative variance: a few "difficult" points or labels can disproportionately affect which splits look best, so the scheduler may alternate between "stable" and "unstable" modes, producing less stable speed/accuracy trade-offs.

**Noisy embeddings (CoLES) and robustness (speculation).** On *Age Prediction* (CoLES embeddings), the consistent gains suggest that HASBoost may cope better with noisy or weakly-informative features. A plausible explanation is that delaying approximation until a stable regime and then focusing sketching on persistent, high-curvature/high-gradient "anchor" cases acts as an implicit denoising/regularization mechanism: late-stage micro-corrections to idiosyncratic embedding noise are suppressed, while the model capacity is steered toward more stable signal.

## 8. Limitations

While HASBoost demonstrates significant advantages in accelerating training and inference for multi-output tasks with limited quality degradation, several limitations should be noted. First, its adaptive sampling mechanism introduces a stochastic component, which can lead to variability in the trained model across different runs. This may be undesirable in applications requiring strict reproducibility. Second, the method involves computational overhead for maintaining gradient history and computing sampling weights. This overhead may not be justified on small datasets or when the number of selected outputs is large relative to the total, making traditional boosting more efficient. Finally, the approach assumes some correlation among output variables to benefit from joint sketching; it may be less effective if outputs are completely independent.

## 9. Conclusion

We introduced HASBoost, an adaptive gradient boosting algorithm tailored for high-dimensional multi-output problems. By leveraging per-sample optimization history and a lightweight change-point scheduler, HASBoost dynamically activates joint row–column sketching of gradient and Hessian matrices only after detecting stable training regimes. This approach substantially reduces computational overhead while preserving — and in many cases improving — predictive performance.

Empirical results (Table 3) show regime-dependent behavior: in larger/high-cost settings, HASBoost can deliver substantial training-time gains (up to 9.6× versus XGBoost), while on smaller-scale tasks the speedup can diminish or reverse due to overhead. In the largest/high-cost regimes, HASBoost matches or improves the best F1 while substantially reducing wall-clock time: on *MNIST*, training drops from 121,399 ms (XGBoost) to 28,918 ms (HASBoost) with F1 reaching 0.98; on *MediaMill*, training drops from 138,563 ms to 14,471 ms while achieving the best/tied-best F1 of 0.58. On *Age Prediction*, HASBoost improves F1 from 0.23 (XGBoost) to 0.38 while also reducing training time from 1,354 ms to 567 ms, suggesting that history-gated sketching can yield both accuracy and efficiency gains. Compared to SketchBoost, HASBoost reduces training time by 1.3–8.1× and inference time by 1.1–7.8× depending on the dataset, while matching or improving F1.

Theoretically, we provide an interpretation of the implicit regularization induced by history-guided sketching and show that split-score approximations admit simple error controls (including a biased truncation view and an unbiased alternative estimator). This helps explain the observed tendency toward simpler (shallower, wider) trees and improved generalization without explicit complexity penalties.

*Table 3.* Main results.

| Dataset | Metric | Sigmoid | Linear | Baselines | LightGBM | XGBoost |
|---|---|---|---|---|---|---|
| age_prediction | f1 | $0.362 \pm 0.003$ (-0.146) | $\mathbf{0.364 \pm 0.004}$ (+0.099) | $0.363 \pm 0.004$ | $0.293 \pm 0.007$ (-6.992) | $0.293 \pm 0.006$ (-7.024) |
| | inference_time | $\mathbf{19.101 \pm 0.873}$ (-57.3%) | $21.104 \pm 1.641$ (-52.8%) | $44.756 \pm 2.881$ | $126.862 \pm 1.324$ (+183.5%) | $98.493 \pm 2.441$ (+120.1%) |
| | train_time | $7555.158 \pm 454.636$ (+7.9%) | $5833.342 \pm 406.882$ (-16.7%) | $7000.000 \pm 300.000$ | $4031.379 \pm 35.788$ (-42.4%) | $8286.741 \pm 138.380$ (+18.4%) |
| cifar10 | f1 | $0.445 \pm 0.004$ (+0.001) | $\mathbf{0.470 \pm 0.004}$ (+2.501) | $0.445 \pm 0.004$ | $0.265 \pm 0.006$ (-17.964) | $0.255 \pm 0.005$ (-18.959) |
| | inference_time | $\mathbf{12.138 \pm 0.260}$ (-0.0%) | $12.805 \pm 0.260$ (+5.5%) | $\mathbf{12.138 \pm 0.260}$ | $119.533 \pm 1.988$ (+884.8%) | $53.106 \pm 1.390$ (+337.5%) |
| | train_time | $\mathbf{1008.633 \pm 16.469}$ (-0.0%) | $1064.107 \pm 16.469$ (+5.5%) | $\mathbf{1008.633 \pm 16.469}$ | $6602.212 \pm 86.107$ (+554.6%) | $20011.505 \pm 90.475$ (+1884.0%) |
| mediamill | f1 | $\mathbf{0.636 \pm 0.001}$ (+1.242) | $\mathbf{0.636 \pm 0.001}$ (+1.233) | $0.624 \pm 0.001$ | $0.612 \pm 0.010$ (-1.200) | $0.608 \pm 0.011$ (-1.600) |
| | inference_time | $161.052 \pm 26.332$ (-85.0%) | $\mathbf{111.886 \pm 10.593}$ (-89.6%) | $1076.787 \pm 118.010$ | $5484.543 \pm 53.485$ (+409.3%) | $2308.792 \pm 40.465$ (+114.4%) |
| | train_time | $28408.017 \pm 7495.556$ (-85.1%) | $\mathbf{13589.973 \pm 2511.710}$ (-92.9%) | $190683.540 \pm 8163.140$ | $98058.349 \pm 935.113$ (-48.6%) | $198613.332 \pm 1024.973$ (+4.2%) |
| MoA | f1 | $0.642 \pm 0.0021$ (-0.0016) | $\mathbf{0.646 \pm 0.0019}$ (+0.0018) | $0.644 \pm 0.0020$ | $0.633 \pm 0.0023$ (-0.0112) | $0.630 \pm 0.0021$ (-0.0141) |
| | inference_time | $68.3 \pm 1.3$ (-7.8) | $\mathbf{64.9 \pm 1.5}$ (-12.4) | $74.1 \pm 2.0$ | $208.0 \pm 3.4$ (+133.9) | $191.0 \pm 2.8$ (+118.0) |
| | train_time | $1610 \pm 45$ (-0.094) | $\mathbf{1490 \pm 38}$ (-0.162) | $1777 \pm 52$ | $2220 \pm 90$ (+0.249) | $2074 \pm 75$ (+0.167) |
| Delicious | f1 | $\mathbf{0.412 \pm 0.006}$ (+0.0017) | $0.409 \pm 0.005$ (-0.0013) | $0.410 \pm 0.005$ | $0.338 \pm 0.010$ (-0.0716) | $0.342 \pm 0.009$ (-0.0679) |
| | inference_time | $132.0 \pm 3.0$ (-10.5) | $\mathbf{126.5 \pm 2.8}$ (-14.2) | $147.5 \pm 3.5$ | $465 \pm 10$ (+215.3) | $418 \pm 9$ (+183.4) |
| | train_time | $18500 \pm 520$ (-0.052) | $16800 \pm 480$ (-0.140) | $19550 \pm 600$ | $15900 \pm 420$ (-0.187) | $\mathbf{15795 \pm 380}$ (-0.192) |

Results are reported as mean $\pm$ standard deviation (difference vs. baseline in parentheses).

Despite these advances, several limitations remain: the scheduler's reliance on finite differences introduces noise sensitivity, per-sample history incurs modest memory overhead at extreme scale, and the diagonal Hessian approximation may discard valuable cross-output information. Future work will explore more robust change-point detection, memory-efficient history compression, structured Hessian approximations, and evaluation on high-cardinality and non-stationary multi-output tasks.

Overall, HASBoost demonstrates that history-guided, adaptive approximations can reduce split-search cost within sequential gradient boosting for multi-output settings, opening promising directions for scalable, theoretically grounded ensemble methods in high-dimensional output spaces.

## Impact Statement

Faster training can reduce the barrier to deploying multi-output models (e.g., multi-label classifiers) under limited compute budgets and can shorten iteration cycles during model development. However, adaptive schemes that prioritize "difficult" examples or labels may amplify spurious patterns if difficulty is driven by noise, outliers, or measurement error. Accordingly, while HASBoost shows strong accuracy–efficiency trade-offs on standard benchmarks, systematic evaluation of fairness, label-noise robustness, and worst-case subgroup behavior remains important future work.

## References


Breiman, L. Bagging predictors. *Machine Learning*, 24(2): 123–140, 1996.

Calzavara, S., Cazzaro, L., Lucchese, C., and Pibiri, G. E. Verifiable boosted tree ensembles. In *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 2057–2075, 2025. doi: 10.1109/SP61157.2025.00022.

Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794, 2016.

Emine, Y., Forel, A., Malek, I., and Vidal, T. Free lunch in the forest: Functionally-identical pruning of boosted tree ensembles. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 16488–16495, 2025. doi: 10.1609/aaai.v39i16.33811.

Freund, Y. and Schapire, R. E. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

Friedman, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

Friedman, J. H. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002.

Iosipoi, A., Vogels, T., Sakr, C., and Jaggi, M. SketchBoost: Fast gradient boosted decision trees for multioutput problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Lindskog, W., Prehofer, C., and Singh, S. Histogram-based federated xgboost using minimal variance sampling for federated tabular data. In *2023 Eighth International Conference on Fog and Mobile Edge Computing (FMEC)*, pp. 182–189. IEEE, September 2023. doi: 10.1109/fmec59375.2023.10306242. URL http://dx.doi.org/10.1109/FMEC59375.2023.10306242.

Mason, L., Baxter, J., Bartlett, P. L., and Frean, M. R. Boosting algorithms as gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*. 1999.

Pezoulas, V. C., Kalatzis, F., Exarchos, T. P., Goules, A., Tzioufas, A. G., and Fotiadis, D. I. Fhbf: Federated hybrid boosted forests with dropout rates for supervised learning tasks across highly imbalanced clinical datasets. *Patterns*, 5(1):100893, 2024. doi: 10.1016/j.patter.2023.100893.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Rashmi, K. N. and Gilad-Bachrach, R. DART: Dropouts meet multiple additive regression trees. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 789–797, 2015.

Zhang, F., Wang, L., Cui, C., Meng, Q., and Yang, M. Evfex: An efficient vertical federated xgboost algorithm based on optimized secure matrix multiplication. *Signal Processing*, 227:109686, 2025. ISSN 0165-1684. doi: https://doi.org/10.1016/j.sigpro.2024.109686. URL https://www.sciencedirect.com/science/article/pii/S0165168424003062.

Zhang, T. and Yu, B. Boosting with early stopping: Convergence and consistency. *The Annals of Statistics*, 33(4): 1538–1572, 2005.

# Appendix

## A. Implicit Tikhonov Regularization via Adaptive Sketching (details)

In the stable regime defined by (9), the global volatility of per-sample gradient norms is small by construction. This reduces the variability of split statistics under row–column subsampling and makes the induced approximation noise more controlled.

Standard sketching and concentration results (e.g., for randomized low-rank approximations and importance sampling) imply that, under mild boundedness assumptions on per-split sufficient statistics, the deviation between exact and sketched split scores can be bounded with high probability; moreover, the required sketch size decreases as the underlying dynamics stabilize.

**Lemma A.1** (Sketched split-score error (biased vs. unbiased))**.** *Fix a leaf region $R$ and define the exact per-output contribution*

$$s_k(R) := \frac{\left(\sum_{i\in R} g_i^k\right)^2}{\sum_{i\in R} h_i^k + \lambda}, \qquad k = 1, \dots, d, \tag{15}$$

*so that $S(R) = \sum_{k=1}^{d} s_k(R)$. Consider the column-sketched score computed on a subset $\mathcal{J}$ of size $c$ without reweighting,*

$$\widetilde{S}(R) := \sum_{k\in\mathcal{J}} s_k(R). \tag{16}$$

*Then $\widetilde{S}(R)$ is generally* biased *(it underestimates $S(R)$), and its deterministic approximation error satisfies*

$$0 \le S(R) - \widetilde{S}(R) = \sum_{k\notin\mathcal{J}} s_k(R) \le (d-c) \max_{k\in[d]} s_k(R). \tag{17}$$

*An (approximately)* unbiased *alternative is the Horvitz–Thompson estimator based on column sampling probabilities $\{q_k\}_{k=1}^{d}$:*

$$\widehat{S}(R) := \sum_{k\in\mathcal{J}} \frac{s_k(R)}{q_k}, \tag{18}$$

*which trades bias for increased variance. In particular, the variance contribution of a term scales roughly as $s_k(R)^2/q_k$, so choosing $q_k$ to upweight large-contribution outputs (as encouraged by our history-guided $W^{(t)}$) reduces estimator variance compared to uniform sampling.*

*Remark* A.2 (Extension to joint row–column sketching). Lemma A.1 states the idea for column sketching. The full HASBoost procedure also subsamples rows. Row-subsetting induces analogous approximation effects for the sufficient statistics $\sum_{i\in R} g_i^k$ and $\sum_{i\in R} h_i^k$ used in $s_k(R)$, hence the same style of (biased) truncation bound applies after replacing the exact sums by their row-sketched counterparts. An unbiased alternative can be obtained by standard importance reweighting of sampled rows (Horvitz–Thompson), at the expense of larger variance.

**Proposition A.3** (Margin implies correct split selection)**.** *Consider a fixed node and let $\mathrm{Gain}^\star$ and $\mathrm{Gain}^{(2)}$ denote the best and second-best exact gains among candidate splits. Assume the gain estimator $\widetilde{\mathrm{Gain}}$ (computed using the sketch) satisfies a uniform error bound $|\widetilde{\mathrm{Gain}} - \mathrm{Gain}| \le \delta$ for all candidate splits. If the margin satisfies $\mathrm{Gain}^\star - \mathrm{Gain}^{(2)} > 2\delta$, then the maximizer of $\widetilde{\mathrm{Gain}}$ coincides with the maximizer of $\mathrm{Gain}$. In practice, one may estimate $\delta$ empirically by periodically evaluating exact gains on a small random subset of candidate splits and taking the maximum observed deviation.*

In HASBoost, the scheduler in Eq. 9 activates sketching only in stable regimes, where empirical volatility is low. In this setting, the effective gain-approximation error $\delta$ is typically smaller; for the biased truncation estimator, Eq. 17 provides a simple worst-case upper bound.

Selective sketching in stable phases filters high-frequency components of updates—local adjustments that often overfit noise—effectively applying a low-pass filter to the boosting trajectory. By initiating row-column sketching only after EMA-smoothed dynamics stabilize, our method biases the ensemble toward smoother functions in the tree span $\mathcal{H}$.

This implicit regularization resembles early stopping in gradient descent (Zhang & Yu, 2005): by compressing gradient/Hessian information in stable regions, we attenuate late-stage, high-variance micro-corrections and tend to favor simpler

decision boundaries. The scheduler parameter $\theta$ (and smoothing factor $\alpha$) controls when approximation begins, modulating the expressive power of subsequent trees—larger $\theta$ triggers earlier sketching, yielding shallower trees and improved generalization (Section 6).

Unlike fixed early stopping or explicit penalties, our history-guided approach adapts locally per sample and dynamically over iterations, offering a lightweight, online mechanism to trade computation for model complexity. In multi-output settings, this enables output-compressed updates where tasks share stable dynamics, scaling efficiently without degrading accuracy.

## B. Sketching details

**Horvitz–Thompson (unbiased) reweighting.** In ablations, we consider unbiased estimators via standard Horvitz–Thompson reweighting for sampled rows (using $p_i^{(t)}$) and/or sampled columns (using $q_j^{(t)}$), trading bias for increased variance.

**Sketching variants.** We use non-uniform sampling without replacement as the default selection mechanism. For completeness, we also consider a greedy alternative that selects the top-$\lfloor rn \rfloor$ rows by $p_i^{(t)}$ and top-$c$ columns by $q_j^{(t)}$.

**HASBoost-SVD implementation note.** The rank $k$ of the approximation is computed as the number of nonzero singular values.

**Contribution of Weighting and Sketching** We evaluate whether the added weighting computations introduce a meaningful runtime penalty during training. Comparing the linear and sigmoidal weighting variants shows (at Fig. 3) that this extra cost is small (at most about 3% of total training time). Given the substantially larger acceleration obtained from sketch-based ensemble compression, the weighting overhead is negligible in the overall efficiency budget.

*Table 4.* Contribution of weighting variants to sketching overhead (mean $\pm$ std).

| Metric | Linear | Sigmoid |
|---|---|---|
| Sketch proportion, % | $1.25 \pm 0.35$ | $1.33 \pm 0.36$ |
| Train time (ms) | $4371.70 \pm 1148.70$ | $3252.91 \pm 1353.56$ |

## C. HASBoost Hyperparameters Tuning

### C.1. Experimental Setup and Baseline Configuration

For the analogue methods, which include LightGBM and XGBoost, the hyperparameter search was designed to explore a limited set of key parameters known to influence model performance. The learning rate was allowed to take values of $0.1$ and $0.005$, providing a balance between relatively fast convergence and more stable optimization. The subsample ratio, which controls the fraction of training data used for each boosting iteration, was explored at three levels: $0.05$, $0.5$, and $0.75$. This range allows us to evaluate aggressive subsampling against more conservative strategies.

All other hyperparameters were held constant to ensure a fair comparison across experiments. Specifically, the feature fraction was set to $0.75$, the maximum tree depth was fixed at $6$, the L1 and L2 regularization parameters were both set to $1$, the minimum number of samples required in a leaf node was set to $50$, and the number of parallel jobs was set to $8$.

For the sigmoid and hyperbolic weighting methods, the hyperparameter search space was expanded to account for the additional complexity introduced by the weighting mechanisms. The learning rate was again explored at $0.1$ and $0.005$, while the subsample ratio was evaluated at five levels: $0.05$, $0.25$, $0.5$, $0.75$, and $1.0$.

The smoothing parameter $\alpha$, which controls the exponential moving average of gradient and Hessian norms, was explored at values of $0.7$, $0.8$, and $0.9$. The stabilization threshold parameter $\theta$, which determines when sketching is activated based on gradient norm stability, was evaluated at $0.5$, $1.0$, and $2.0$.

The sketching method was fixed to the `topk` strategy. The number of sketch outputs was dynamically defined as the product of the number of classes and the subsample ratio, rounded to the nearest integer with a minimum of one. This ensures that the sketch size scales proportionally with both the problem dimensionality and the sampling rate.

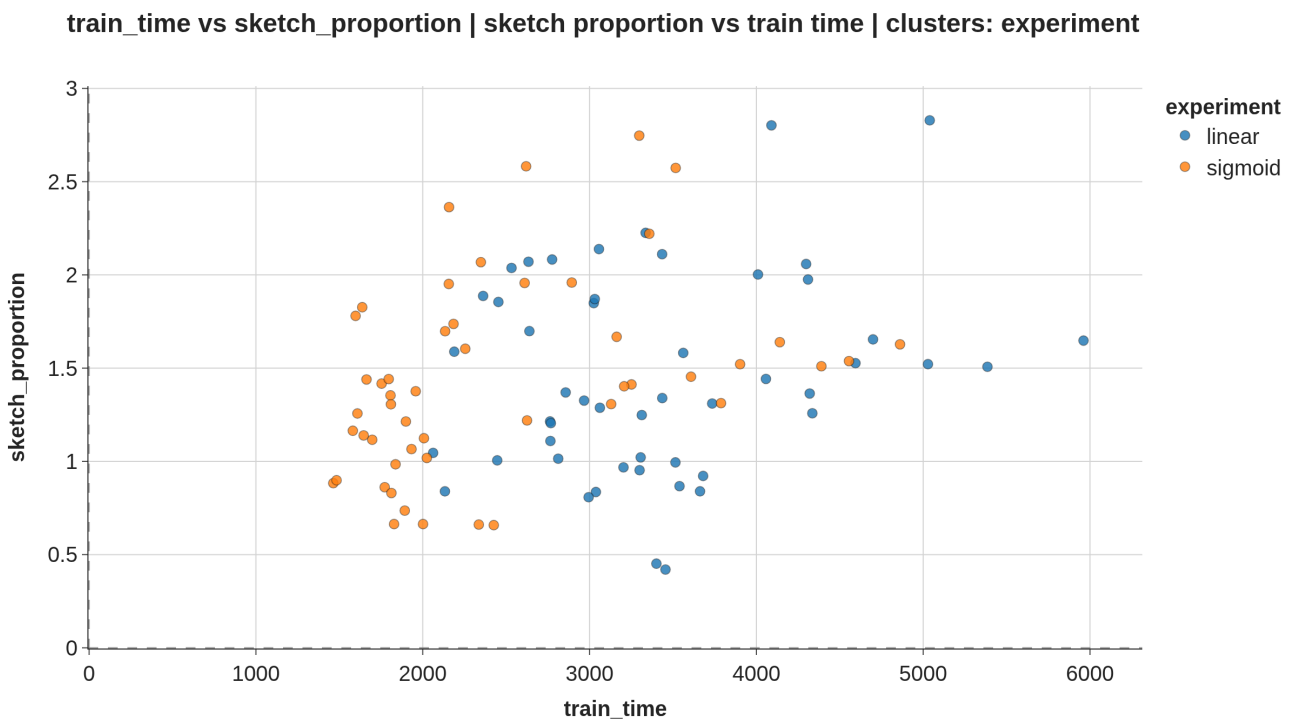


*Figure 3.* Contribution of HASBoost computation overhead to train time across all possible hyperparameters combinations.

Finally, early stopping with a patience of 15 iterations and a maximum of 200,000 trees were used to provide sufficient capacity for convergence while preventing overfitting.

### C.2. HASBoost Hyperparameter Sweep

To study the impact of scheduler hyperparameters, we ran a controlled sweep over the stabilization threshold $\theta$ and the EMA smoothing factor $\alpha$. We report relative performance with respect to SketchBoost under matched row/column budgets.

The search grid is $\theta \in \{1, 2\}$, $\alpha \in \{0.7, 0.8, 0.9\}$, $r \in \{0.05, 0.25, 0.5, 0.75, 1.0\}$, and $c \in \{0.05, 0.25, 0.5, 0.75, 1.0\}$, where $r$ and $c$ denote row/column sketching proportions. Experiments in this subsection use CIFAR10. For each configuration, we compare HASBoost against SketchBoost with the same $(r, c)$ and report metric differences.

**Stabilization threshold** The stabilization threshold controls when sketching is activated and therefore sets a direct speed–accuracy trade-off.

Figure 4 indicates that smaller $\theta$ values tend to yield higher accuracy but lower acceleration, while larger $\theta$ values favor faster training/inference with a modest quality drop. This behavior is consistent with earlier sketch activation: when approximation starts before the ensemble has captured dominant structure, split estimates become noisier.

The plot suggests three practical operating regimes as $\theta$ increases: a conservative regime ($\theta \approx 0.5$) with near-baseline quality and modest speedup, a balanced regime ($\theta \in [0.5, 1.0]$) with substantial acceleration and moderate quality loss, and an aggressive regime ($\theta > 1.0$) with only marginal additional speed gains but pronounced F1 degradation. Therefore, $\theta \in [0.5, 1.0]$ is a robust default range, with the exact choice determined by whether accuracy or runtime is prioritized.

**Smoothing alpha** The parameter $\alpha$ determines how strongly historical gradient statistics are retained in the EMA. Larger $\alpha$ increases inertia in the stability signal and typically delays changes in sketching intensity. We evaluate this effect across the full hyperparameter grid.

Across all configurations (Fig. 5), we do not observe a strong global monotonic trend between $\alpha$ and final performance.

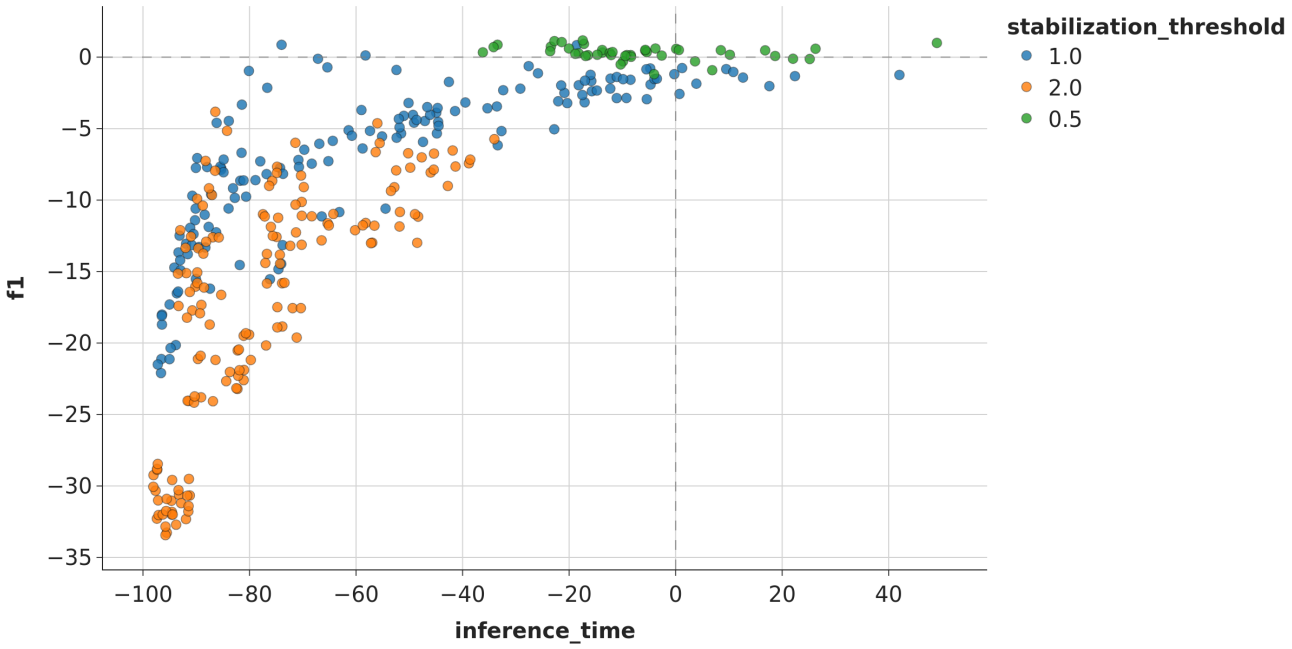


*Figure 4.* Distribution of F1 metric corresponding to inference acceleration for all possible hyperparameter combinations relatively to SketchBoost. The coloration reflects different $\theta$ values.

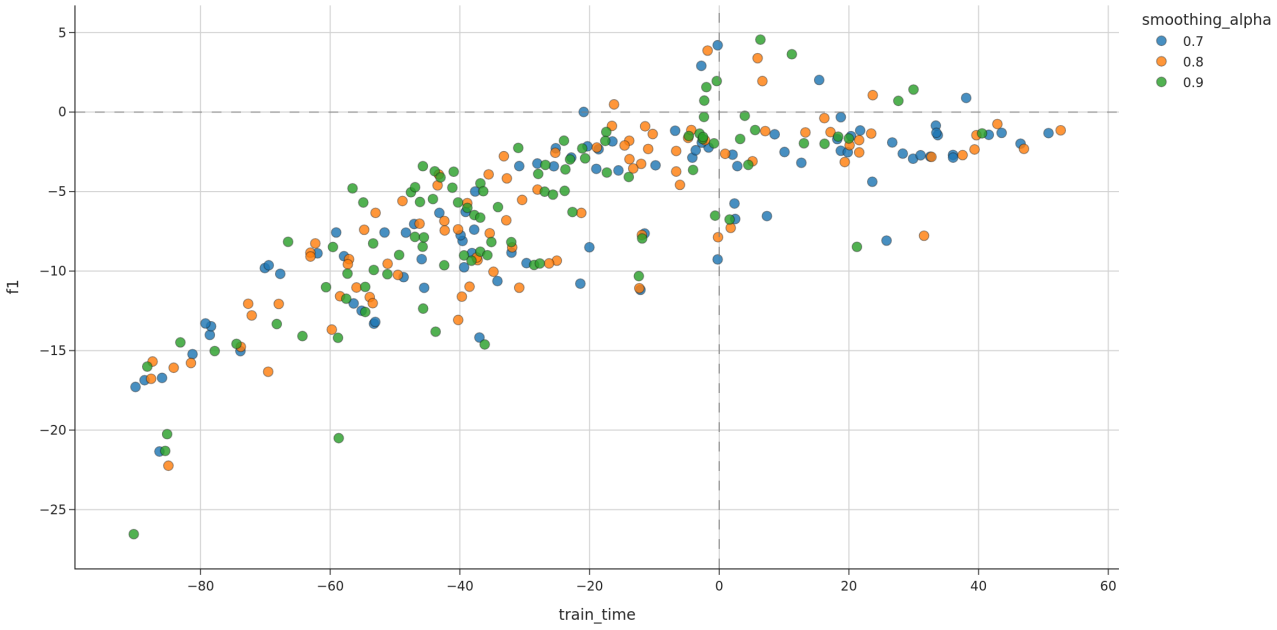


*Figure 5.* Distribution of F1 metric corresponding to train acceleration for all possible hyperparameter combinations relatively to SketchBoost. The coloration reflects different $\alpha$ values.

However, when the column budget is small, the effect becomes clearer (Fig. 6): larger $\alpha$ is associated with a more aggressive sketched regime and higher variance in the speed–quality outcome.

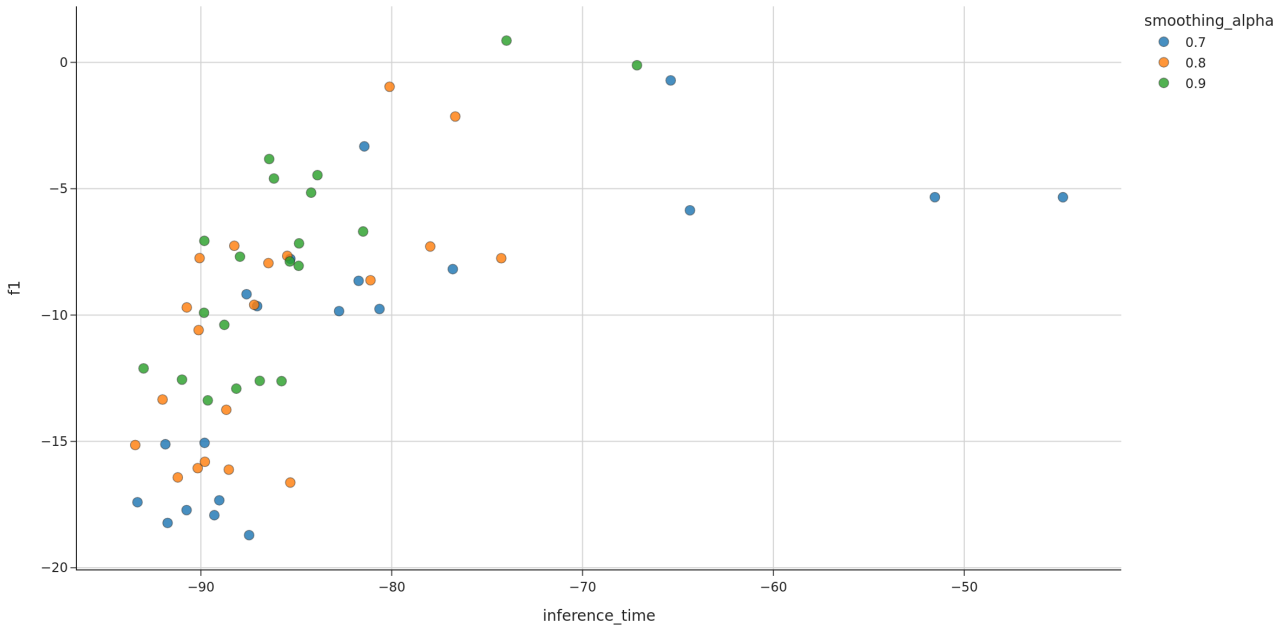


*Figure 6.* Distribution of F1 metric corresponding to train acceleration for all possible hyperparameter combinations relatively to SketchBoost. The coloration reflects different $\alpha$ values. Selection includes experimental setups with $c \in \{0.05, 0.25\}$.

# D. Appendix: Full Results Table

*Table 5.* Performance comparison across datasets and metrics. Results show mean ± standard deviation, with percentage difference relative to the baseline in parentheses.

| Dataset | Metric | Sigmoid | Linear | Baselines | LightGBM | XGBoost |
|---|---|---|---|---|---|---|
| age_prediction | f1 | 0.362 ± 0.003 (-0.146) | 0.364 ± 0.004 (+0.099) | 0.363 ± 0.004 | 0.293 ± 0.007 (-6.992) | 0.293 ± 0.006 (-7.024) |
| | f1_macro | 0.361 ± 0.003 (-0.150) | 0.364 ± 0.004 (+0.097) | 0.363 ± 0.004 | 0.293 ± 0.007 (-6.935) | 0.293 ± 0.006 (-6.967) |
| | f1_micro | 0.377 ± 0.004 (+0.329) | 0.378 ± 0.004 (+0.506) | 0.373 ± 0.003 | 0.325 ± 0.007 (-4.827) | 0.322 ± 0.006 (-5.165) |
| | inference_time | 19.101 ± 0.873 (-57.3%) | 21.104 ± 1.641 (-52.8%) | 44.756 ± 2.881 | 126.862 ± 1.324 (+183.5%) | 98.493 ± 2.441 (+120.1%) |
| | train_time | 7555.158 ± 454.636 (+7.9%) | 5833.342 ± 406.882 (-16.7%) | 7000.000 ± 300.000 | 4031.379 ± 35.788 (-42.4%) | 8286.741 ± 138.380 (+18.4%) |
| cifar10 | f1 | 0.445 ± 0.004 (+0.001) | 0.470 ± 0.004 (+2.501) | 0.445 ± 0.004 | 0.265 ± 0.006 (-17.964) | 0.255 ± 0.005 (-18.959) |
| | f1_macro | 0.446 ± 0.004 (+0.005) | 0.471 ± 0.004 (+2.505) | 0.446 ± 0.004 | 0.266 ± 0.006 (-17.981) | 0.256 ± 0.005 (-18.976) |
| | f1_micro | 0.449 ± 0.005 (-0.025) | 0.474 ± 0.005 (+2.475) | 0.449 ± 0.005 | 0.252 ± 0.006 (-19.692) | 0.245 ± 0.007 (-20.381) |
| | inference_time | 12.138 ± 0.260 (-0.0%) | 12.805 ± 0.260 (+5.5%) | 12.138 ± 0.260 | 119.533 ± 1.988 (+884.8%) | 53.106 ± 1.390 (+337.5%) |
| | train_time | 1008.633 ± 16.469 (-0.0%) | 1064.107 ± 16.469 (+5.5%) | 1008.633 ± 16.469 | 6602.212 ± 86.107 (+554.6%) | 20011.505 ± 90.475 (+1884.0%) |
| genbase | f1 | 0.814 ± 0.021 (+4.273) | 0.940 ± 0.027 (+16.881) | 0.771 ± 0.009 | 0.808 ± 0.012 (+3.654) | 0.793 ± 0.014 (+2.154) |
| | f1_macro | 0.657 ± 0.011 (+4.054) | 0.746 ± 0.028 (+12.899) | 0.617 ± 0.023 | 0.792 ± 0.013 (+17.517) | 0.777 ± 0.015 (+16.017) |
| | f1_micro | 0.879 ± 0.026 (+8.410) | 0.937 ± 0.015 (+14.193) | 0.795 ± 0.006 | 0.814 ± 0.011 (+1.933) | 0.799 ± 0.013 (+0.433) |
| | inference_time | 19.675 ± 1.850 (-95.8%) | 27.207 ± 1.597 (-94.1%) | 464.661 ± 5.848 | 64.576 ± 0.843 (-86.1%) | 31.611 ± 1.641 (-93.2%) |
| | train_time | 1194.767 ± 246.995 (+16.6%) | 579.266 ± 60.586 (-43.5%) | 1024.994 ± 0.180 | 622.939 ± 9.449 (-39.2%) | 6210.800 ± 246.995 (+505.9%) |
| mediamill | f1 | 0.636 ± 0.001 (+1.242) | 0.636 ± 0.001 (+1.233) | 0.624 ± 0.001 | 0.612 ± 0.010 (-1.200) | 0.608 ± 0.011 (-1.600) |
| | f1_macro | 0.386 ± 0.003 (+5.435) | 0.374 ± 0.004 (+4.316) | 0.331 ± 0.006 | 0.340 ± 0.012 (+0.878) | 0.338 ± 0.013 (+0.678) |
| | f1_micro | 0.665 ± 0.002 (+2.266) | 0.664 ± 0.001 (+2.259) | 0.642 ± 0.001 | 0.000 ± 0.000 (-64.186) | 0.000 ± 0.000 (-64.186) |
| | inference_time | 161.052 ± 26.332 (-85.0%) | 111.886 ± 10.593 (-89.6%) | 1076.787 ± 118.010 | 5484.543 ± 53.485 (+409.3%) | 2308.792 ± 40.465 (+114.4%) |
| | train_time | 28408.017 ± 7495.556 (-85.1%) | 13589.973 ± 2511.710 (-92.9%) | 190683.540 ± 8163.140 | 98058.349 ± 935.113 (-48.6%) | 198613.332 ± 1024.973 (+4.2%) |
| mnist | f1 | 0.921 ± 0.004 (+0.566) | 0.928 ± 0.002 (+1.332) | 0.915 ± 0.004 | 0.858 ± 0.004 (-5.671) | 0.863 ± 0.002 (-5.191) |
| | f1_macro | 0.919 ± 0.003 (+0.450) | 0.927 ± 0.002 (+1.218) | 0.915 ± 0.004 | 0.856 ± 0.004 (-5.912) | 0.861 ± 0.002 (-5.420) |
| | f1_micro | 0.921 ± 0.004 (+0.586) | 0.928 ± 0.002 (+1.343) | 0.915 ± 0.004 | 0.843 ± 0.005 (-7.210) | 0.848 ± 0.001 (-6.667) |
| | inference_time | 7.488 ± 0.319 (-85.0%) | 49.551 ± 8.324 (-0.9%) | 50.000 ± 2.000 | 56.300 ± 1.558 (+12.6%) | 25.699 ± 0.281 (-48.6%) |
| | train_time | 1481.880 ± 50.117 (-71.5%) | 5702.799 ± 673.286 (+9.7%) | 5200.000 ± 120.000 | 2281.134 ± 110.994 (-56.1%) | 7502.011 ± 42.878 (+44.3%) |
| yeast | f1 | 0.603 ± 0.022 (+1.655) | 0.601 ± 0.029 (+1.444) | 0.587 ± 0.024 | 0.415 ± 0.001 (-17.203) | 0.515 ± 0.020 (-7.208) |
| | f1_macro | 0.486 ± 0.007 (+7.265) | 0.485 ± 0.005 (+7.157) | 0.414 ± 0.007 | 0.279 ± 0.000 (-13.489) | 0.333 ± 0.011 (-8.115) |
| | f1_micro | 0.618 ± 0.021 (+1.494) | 0.615 ± 0.028 (+1.154) | 0.603 ± 0.024 | 0.478 ± 0.001 (-12.550) | 0.535 ± 0.022 (-6.777) |
| | inference_time | 20.505 ± 1.421 (-44.4%) | 22.494 ± 3.423 (-39.1%) | 36.912 ± 3.264 | 19.091 ± 0.027 (-48.3%) | 13.583 ± 0.640 (-63.2%) |
| | train_time | 5881.499 ± 295.064 (-63.7%) | 7085.484 ± 837.802 (-56.2%) | 16195.257 ± 1975.638 | 464.711 ± 3.899 (-97.1%) | 1798.038 ± 19.517 (-88.9%) |
| MoA | f1 | 0.642 ± 0.0021 (-0.0016) | 0.646 ± 0.0019 (+0.0018) | 0.644 ± 0.0020 | 0.633 ± 0.0023 (-0.0112) | 0.630 ± 0.0021 (-0.0141) |
| | f1_macro | 0.639 ± 0.0022 (-0.0021) | 0.643 ± 0.0020 (+0.0014) | 0.641 ± 0.0021 | 0.631 ± 0.0024 (-0.0103) | 0.629 ± 0.0022 (-0.0124) |
| | f1_micro | 0.653 ± 0.0018 (-0.0009) | 0.655 ± 0.0017 (+0.0015) | 0.654 ± 0.0018 | 0.642 ± 0.0020 (-0.0121) | 0.640 ± 0.0019 (-0.0137) |
| | inference_time | 68.3 ± 1.3 (-7.8) | 64.9 ± 1.5 (-12.4) | 74.1 ± 2.0 | 208.0 ± 3.4 (+133.9) | 191.0 ± 2.8 (+118.0) |
| | train_time | 1610 ± 45 (-0.094) | 1490 ± 38 (-0.162) | 1777 ± 52 | 2220 ± 90 (+0.249) | 2074 ± 75 (+0.167) |
| Delicious | f1 | 0.412 ± 0.006 (+0.0017) | 0.409 ± 0.005 (-0.0013) | 0.410 ± 0.005 | 0.338 ± 0.010 (-0.0716) | 0.342 ± 0.009 (-0.0679) |
| | f1_macro | 0.398 ± 0.007 (+0.0026) | 0.394 ± 0.006 (-0.0011) | 0.395 ± 0.006 | 0.322 ± 0.011 (-0.0732) | 0.327 ± 0.010 (-0.0684) |
| | f1_micro | 0.431 ± 0.005 (+0.0012) | 0.429 ± 0.004 (-0.0008) | 0.430 ± 0.004 | 0.357 ± 0.009 (-0.0728) | 0.361 ± 0.008 (-0.0687) |
| | inference_time | 132.0 ± 3.0 (-10.5) | 126.5 ± 2.8 (-14.2) | 147.5 ± 3.5 | 465 ± 10 (+215.3) | 418 ± 9 (+183.4) |
| | train_time | 18500 ± 520 (-0.052) | 16800 ± 480 (-0.140) | 19550 ± 600 | 15900 ± 420 (-0.187) | 15795 ± 380 (-0.192) |

Results are formatted as *mean ± standard deviation (percentage difference from baseline)*.
For quality metrics, larger is better; for time metrics, smaller is better.